

Morphology / Gradient



Contents

1 모폴로지

1.1 일단 import

1.2 테스트용 이미지 만들기 함수

1.3 이미지를 디스플레이하는 함수

1.4 이렇게 생긴 아이

1.5 커널하나 만들고

1.6 여러번 돌면서 침식? 시키라고 하면~

1.7 erode

1.8 침식이란

1.9 침식은 이런식으로

1.10 흰색이 얇아짐

1.11 이렇게 점점 얇아짐

1.12 일단 이미지 다시 부르고

1.13 noise 만들고

1.14 noise 를 입힌 글자 이미지

1.15 Dilation 연산

1.16 dilate

1.17 팽창이란

1.18 팽창은 이런식으로~

1.19 오픈닝 연산

1.20 오프닝 연산을 통한 노이즈 제거

1.21 morphologyEx

1.22 이미지 다시 읽고 노이즈 만들고

1.23 noise가 많은 글자 이미지 또 ~

1.24 테스트 이미지 만들려고 고생하는 중~

1.25 아이쿠 만들었다~

1.26 클로징 연산

1.27 크로징 연산을 통한 노이즈 제거

1.28 열기와 닫기의 차이

1.29 다시 새로 시작

1.30 이번에는 그래디언트 연산

2 **그래디언트**

2.1 이미지 세상에서 그래디언트는?

2.2 그래디언트 중 쇼벨 연산자

2.3 이미지 그리는

2.4 이 이미지 하나 불러와서

2.5 x 방향 쇼벨~

2.6 y 방향 그래디언트

2.7 Laplacian 연산

2.8 x, y 방향의 그래디언트를 합치기

2.9 이번에는 threshold 시켜보고

2.10 그래디언트 연산으로 모폴로지도 시켜보고

Morphology / Gradient

Exported from Confluence · <https://pinkwink.atlassian.net>

모폴로지

일단 import

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

✓ 0.2s

Python

테스트용 이미지 만들기 함수

```
def load_img():
    blank_img = np.zeros((600, 600))
    font = cv2.FONT_HERSHEY_SIMPLEX
    cv2.putText(
        blank_img,
        text="ABCDE",
        org=(50, 300),
        fontFace=font,
        fontScale=5,
        color=(255, 255, 255),
        thickness=25,
        lineType=cv2.LINE_AA,
    )
    return blank_img
```

✓ 0.0s

Python

이미지를 디스플레이하는 함수

```
def display_img(img):
    fig = plt.figure(figsize=(12, 10))
    ax = fig.add_subplot(111)
    ax.imshow(img, cmap="gray")
```

✓ 0.0s

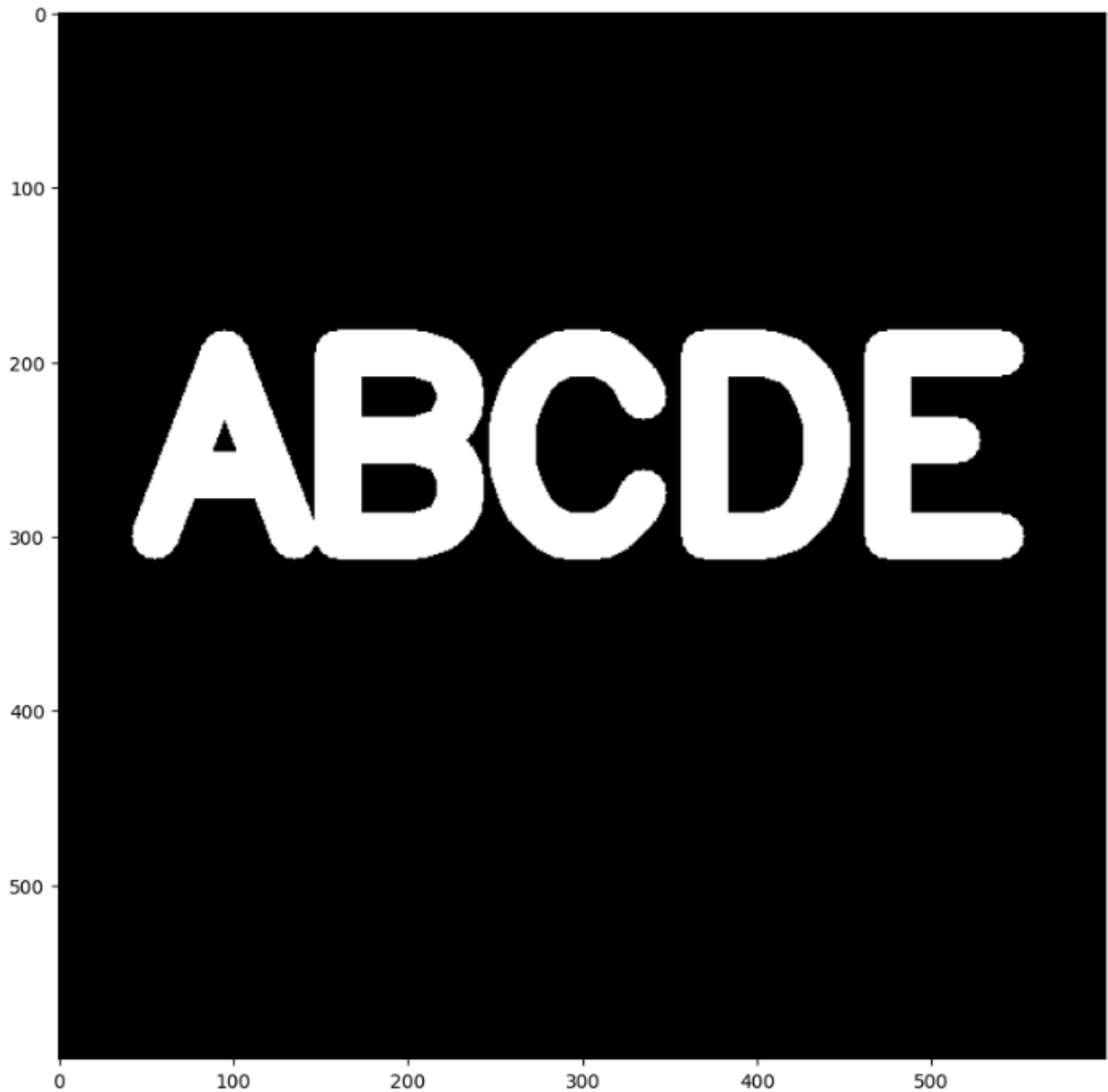
Python

이렇게 생긴 아이

```
img = load_img()  
display_img(img)
```

✓ 0.2s

Python



커널하나 만들고

```
kernel = np.ones((5, 5), dtype=np.uint8)  
kernel
```

✓ 0.0s

Python

```
array([[1, 1, 1, 1, 1],  
       [1, 1, 1, 1, 1],  
       [1, 1, 1, 1, 1],  
       [1, 1, 1, 1, 1],  
       [1, 1, 1, 1, 1]], dtype=uint8)
```

여러번 돌면서 침식? 시키라고 하면~

```
iterations = 5  
result = cv2.erode(img, kernel, iterations=iterations)
```

✓ 0.0s

Python

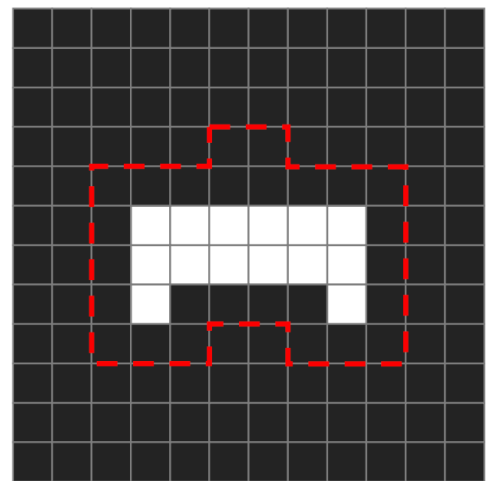
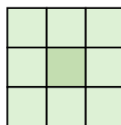
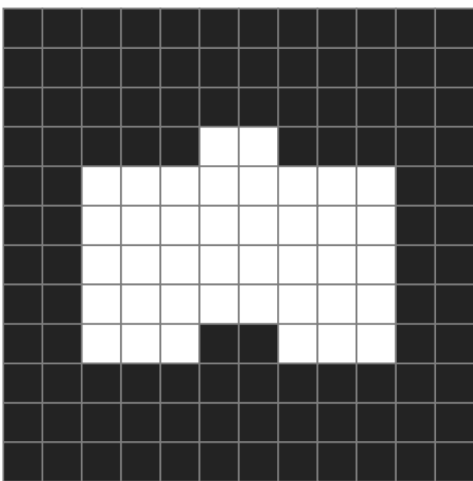
- 바이너리 이미지에서 흰색 오브젝트의 외곽 픽셀을 0(검은색)으로

erode

```
cv2.erode(src, kernel, dst=None, anchor=None, iterations=None,  
borderType=None, borderValue=None) -> dst
```

- src: 입력 영상.
- kernel: 구조 요소. `getStructuringElement()` 함수에 의해 생성 가능. 만약 `None`을 지정하면 3x3 사각형 구성 요소를 사용.
- dst: 출력 영상. src와 동일한 크기와 타입
- anchor: 고정점 위치. 기본값 (-1, -1)을 사용하면 중앙점을 사용.
- iterations: 반복 횟수. 기본값은 1.
- borderType: 가장자리 픽셀 확장 방식. 기본값은 `cv2.BORDER_CONSTANT`.
- borderValue: `cv2.BORDER_CONSTANT`인 경우, 확장된 가장자리 픽셀을 채울 값.

침식이란



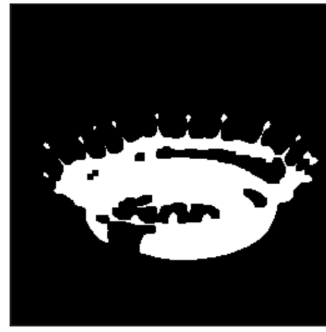
침식은 이런식으로



입력 영상



침식 1회



침식 2회



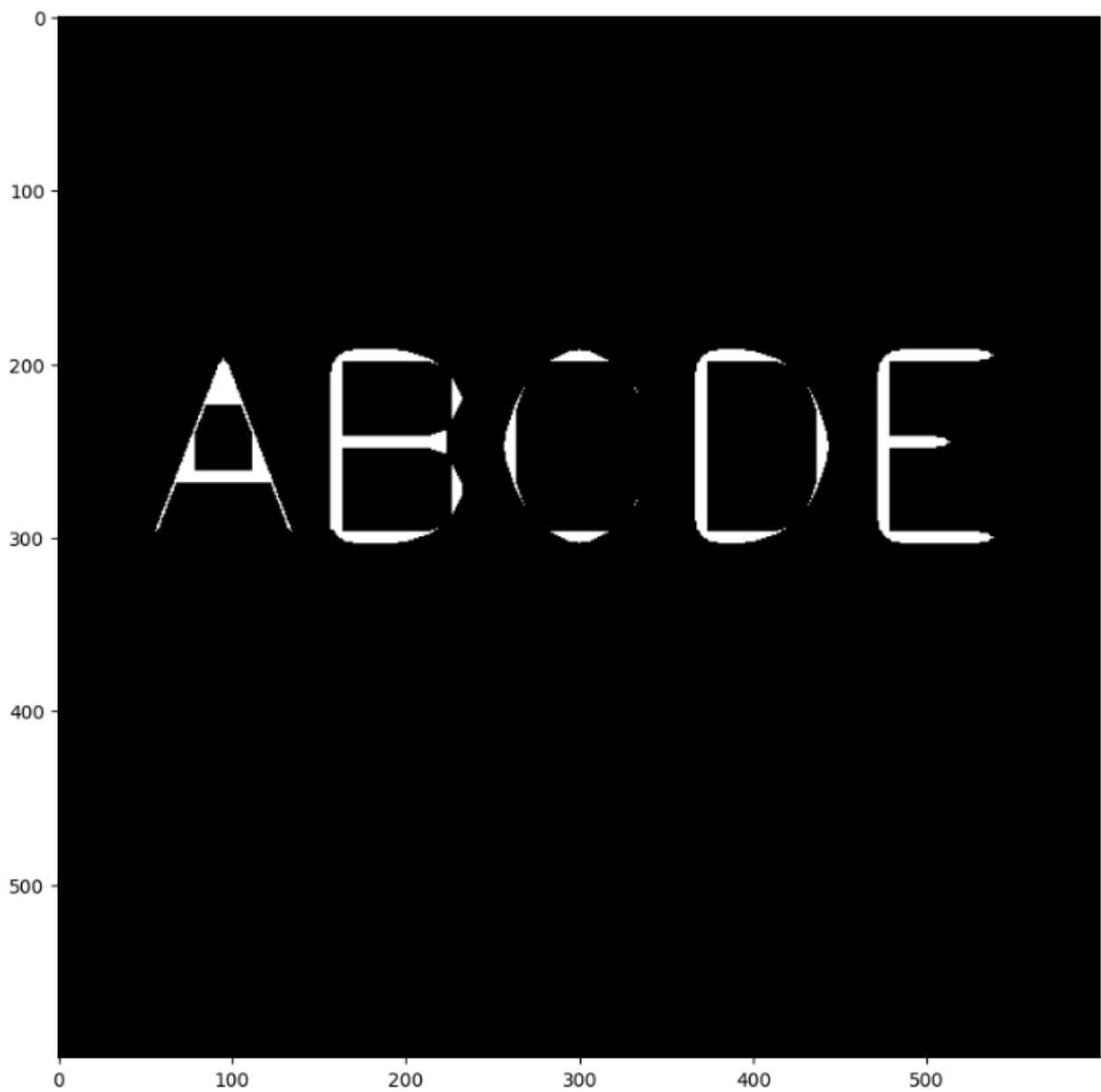
침식 3회

흰색이 얇아짐

```
display_img(result)
```

✓ 0.1s

Python



이렇게 점점 얇아짐



원본 이미지



1 x 1 커널



3 x 3 커널



5 x 5 커널

일단 이미지 다시 부르고

```
img = load_img()
```

✓ 0.0s

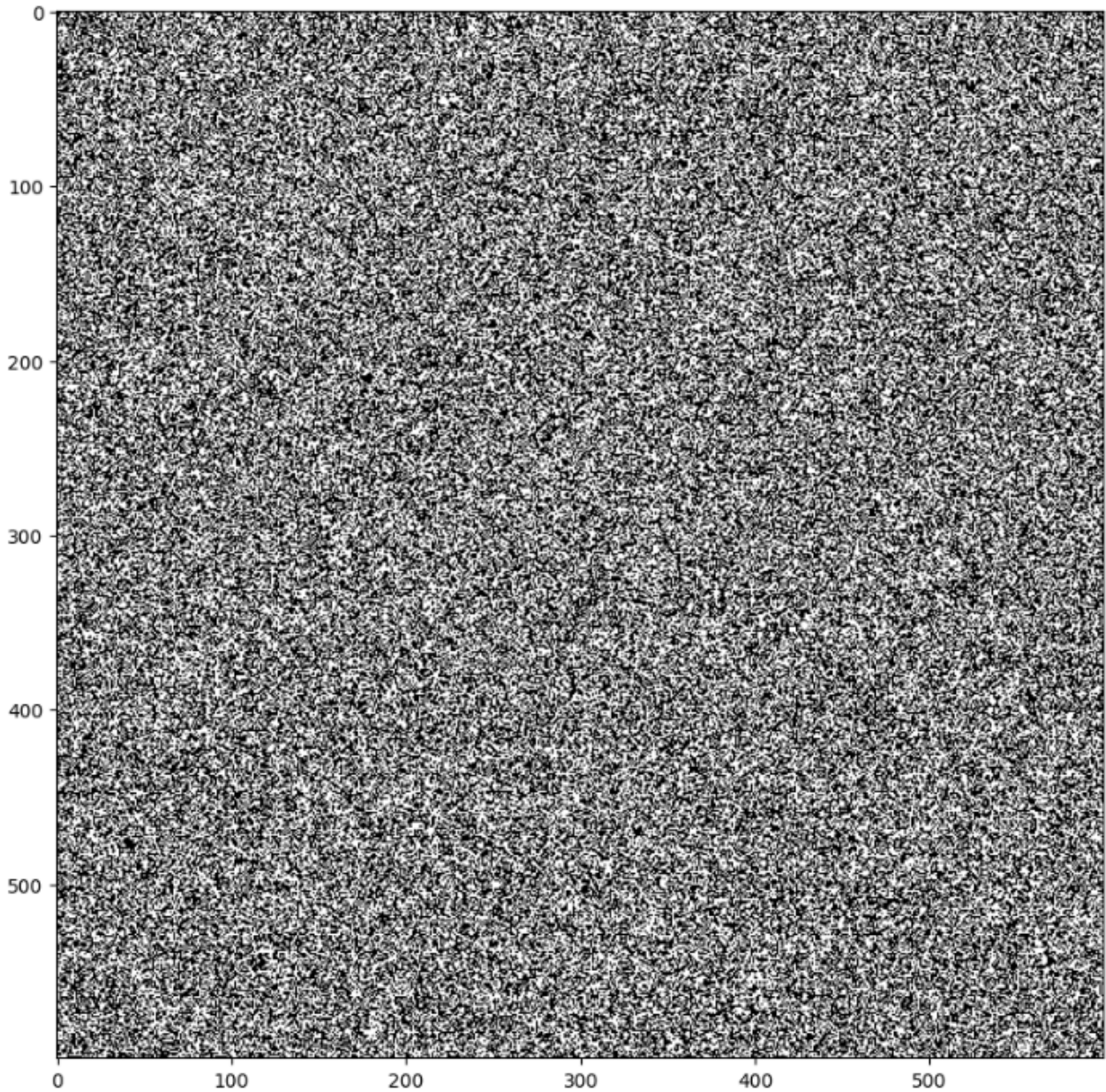
Python

noise 만들고

```
white_noise = np.random.randint(low=0, high=2, size=(600, 600))  
display_img(white_noise)
```

✓ 0.3s

Python

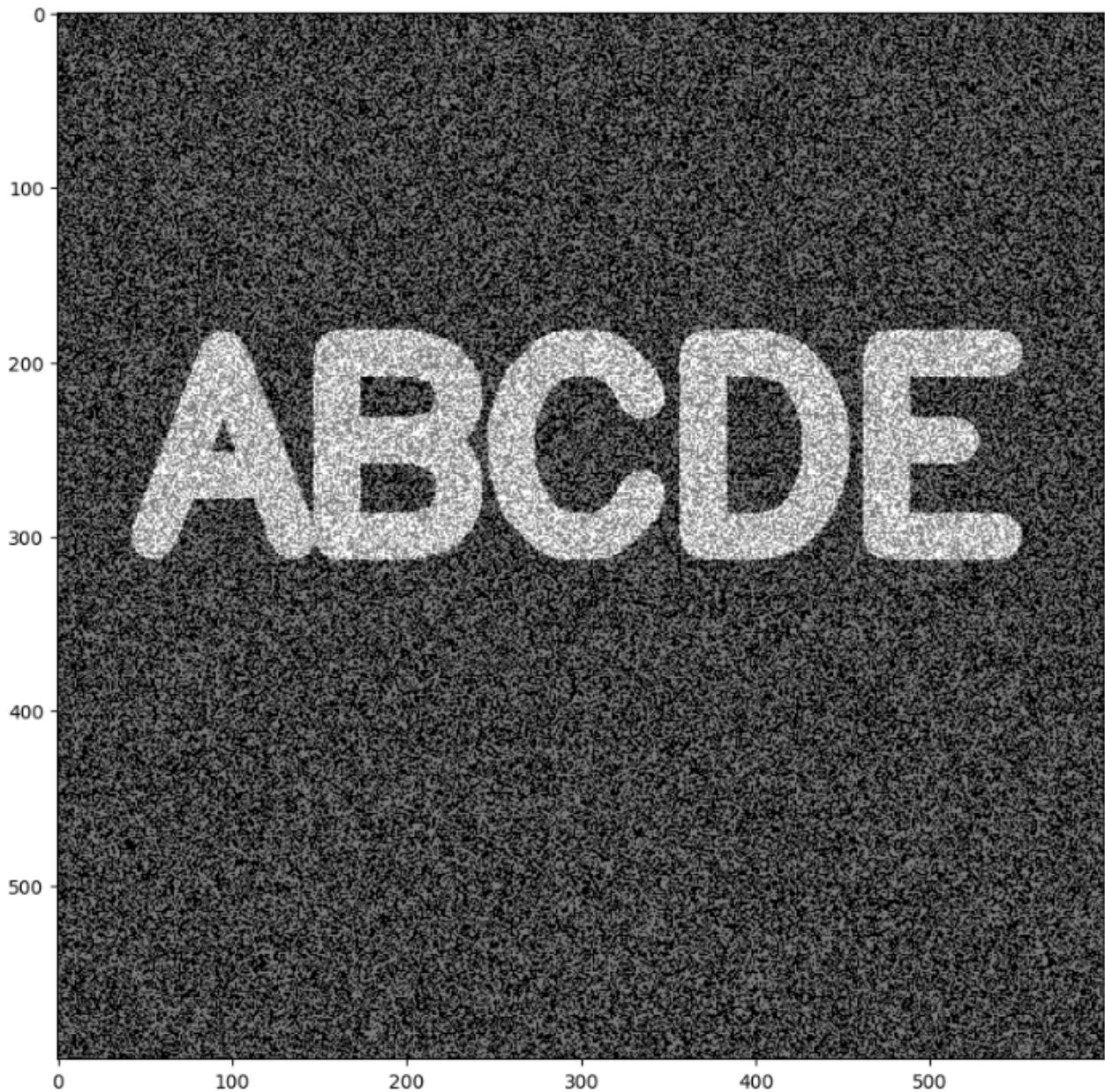


noise 를 입힌 글자 이미지


```
noise_img = white_noise * 255 + img  
display_img(noise_img)
```

✓ 0.4s

Python



Dilation 연산

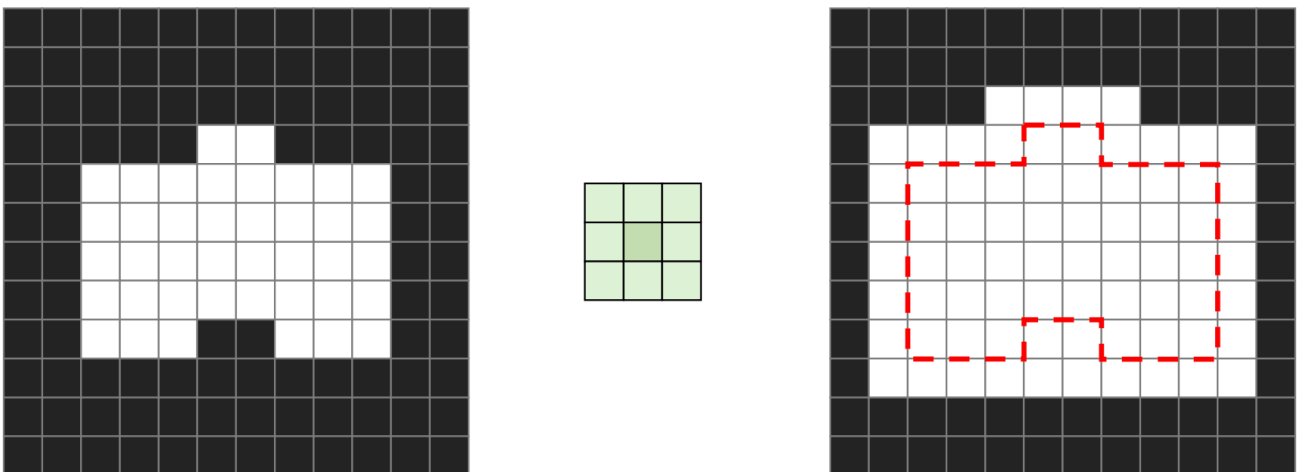
- 바이너리 이미지에서 흰색 오브젝트의 외곽 픽셀 주변에 1(흰색)으로 추가

dilate

```
cv2.dilate(src, kernel, dst=None, anchor=None, iterations=None,
           borderType=None, borderValue=None) -> dst
```

- src: 입력 영상.
- kernel: 구조 요소. `getStructuringElement()` 함수에 의해 생성 가능. 만약 `None`을 지정하면 3x3 사각형 구성 요소를 사용.
- dst: 출력 영상. src와 동일한 크기와 타입
- anchor: 고정점 위치. 기본값 (-1, -1)을 사용하면 중앙점을 사용.
- iterations: 반복 횟수. 기본값은 1.
- borderType: 가장자리 픽셀 확장 방식. 기본값은 `cv2.BORDER_CONSTANT`.
- borderValue: `cv2.BORDER_CONSTANT`인 경우, 확장된 가장자리 픽셀을 채울 값.

팽창이란



팽창은 이런식으로~



입력 영상

팽창 1회

팽창 2회

팽창 3회

오픈링 연산

열기 = 침식 → 팽창

Opening = Erosion → Dilation

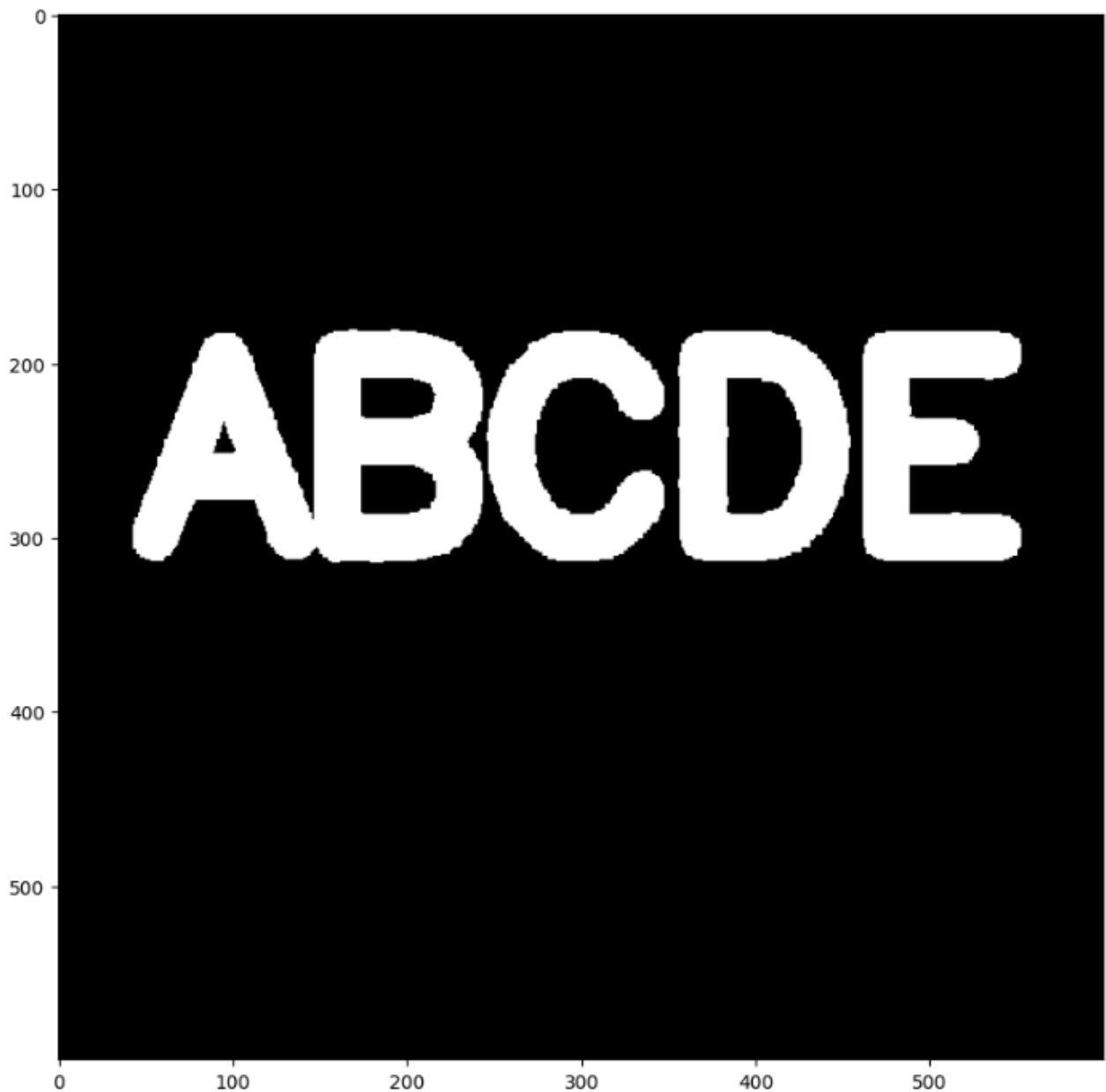
- Erosion 연산 다음에 Dilation 연산을 적용
- 이미지 상의 노이즈(작은 흰색 물체)를 제거하는데 사용

오프닝 연산을 통한 노이즈 제거

```
opening = cv2.morphologyEx(noise_img, cv2.MORPH_OPEN, kernel)  
display_img(opening)
```

✓ 0.2s

Python



morphologyEx

```
cv2.morphologyEx(src, op, kernel, dst=None, anchor=None, iterations=None,
                 borderType=None, borderValue=None) -> dst
```

- src: 입력 영상.
- op: 모폴로지 연산 플래그.

cv2.MORPH_ERODE	침식
cv2.MORPH_DILATE	팽창
cv2.MORPH_OPEN	열기
cv2.MORPH_CLOSE	닫기
cv2.MORPH_GRADIENT	모폴로지 그래디언트 = 팽창 - 침식

- kernel: 커널
- dst: 출력 영상

이미지 다시 읽고 노이즈 만들고


```
img = load_img()
```

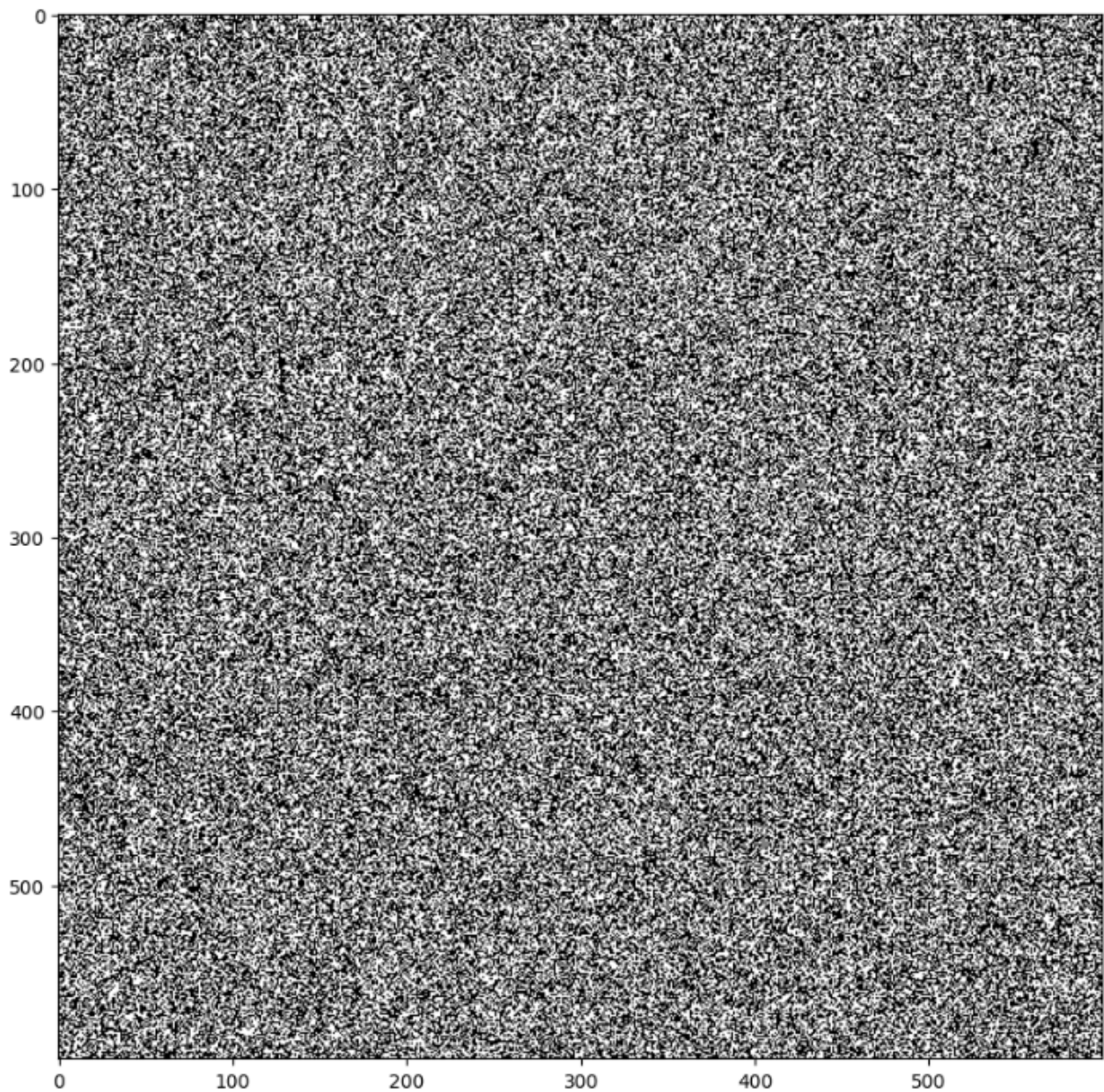
✓ 0.0s

Python

```
black_noise = np.random.randint(low=0, high=2, size=(600, 600))  
black_noise = black_noise * (-img.max())  
display_img(black_noise)
```

✓ 0.3s

Python

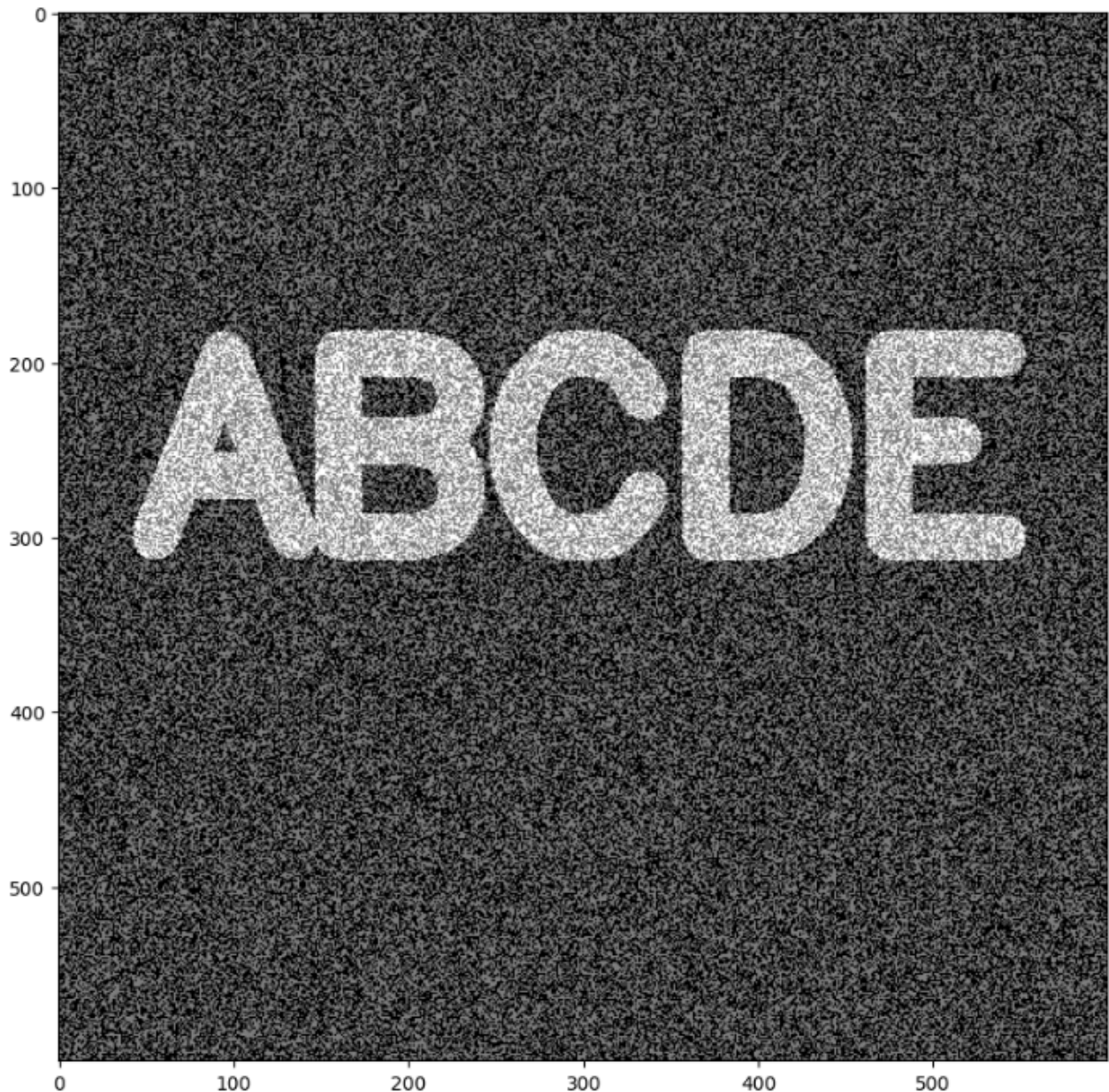


noise가 많은 글자 이미지 또 ~


```
black_noise_img = black_noise + img  
display_img(black_noise_img)
```

✓ 0.3s

Python



테스트 이미지 만들려고 고생하는 중~

```
black_noise_img[black_noise_img == -255] = 0  
black_noise_img.min()
```

✓ 0.0s

Python

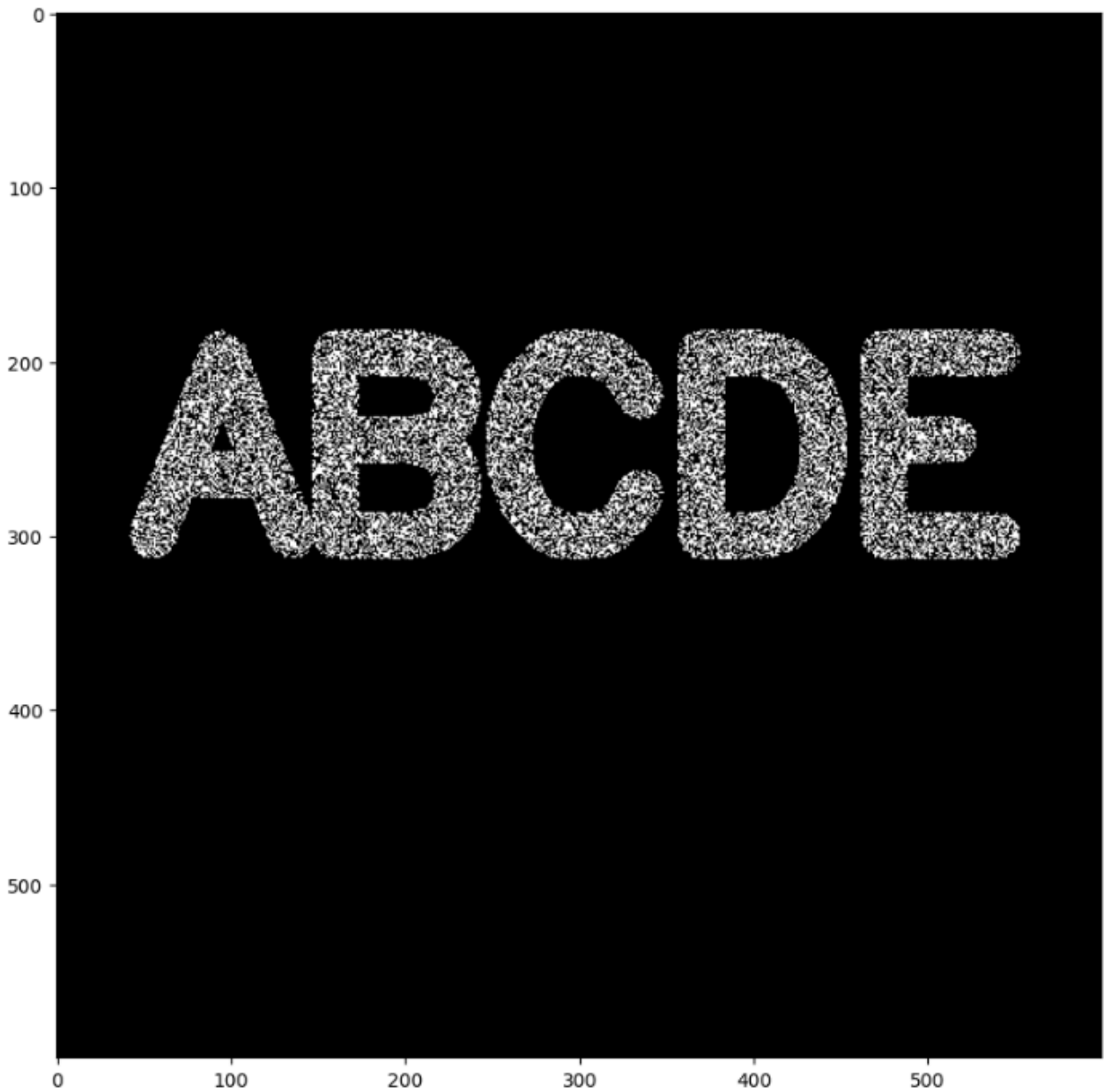
`np.float64(0.0)`

아이쿠 만들었다~

```
display_img(black_noise_img)
```

✓ 0.2s

Python



클로징 연산

닫기 = 팽창 → 침식

Closing = Dilation → Erosion

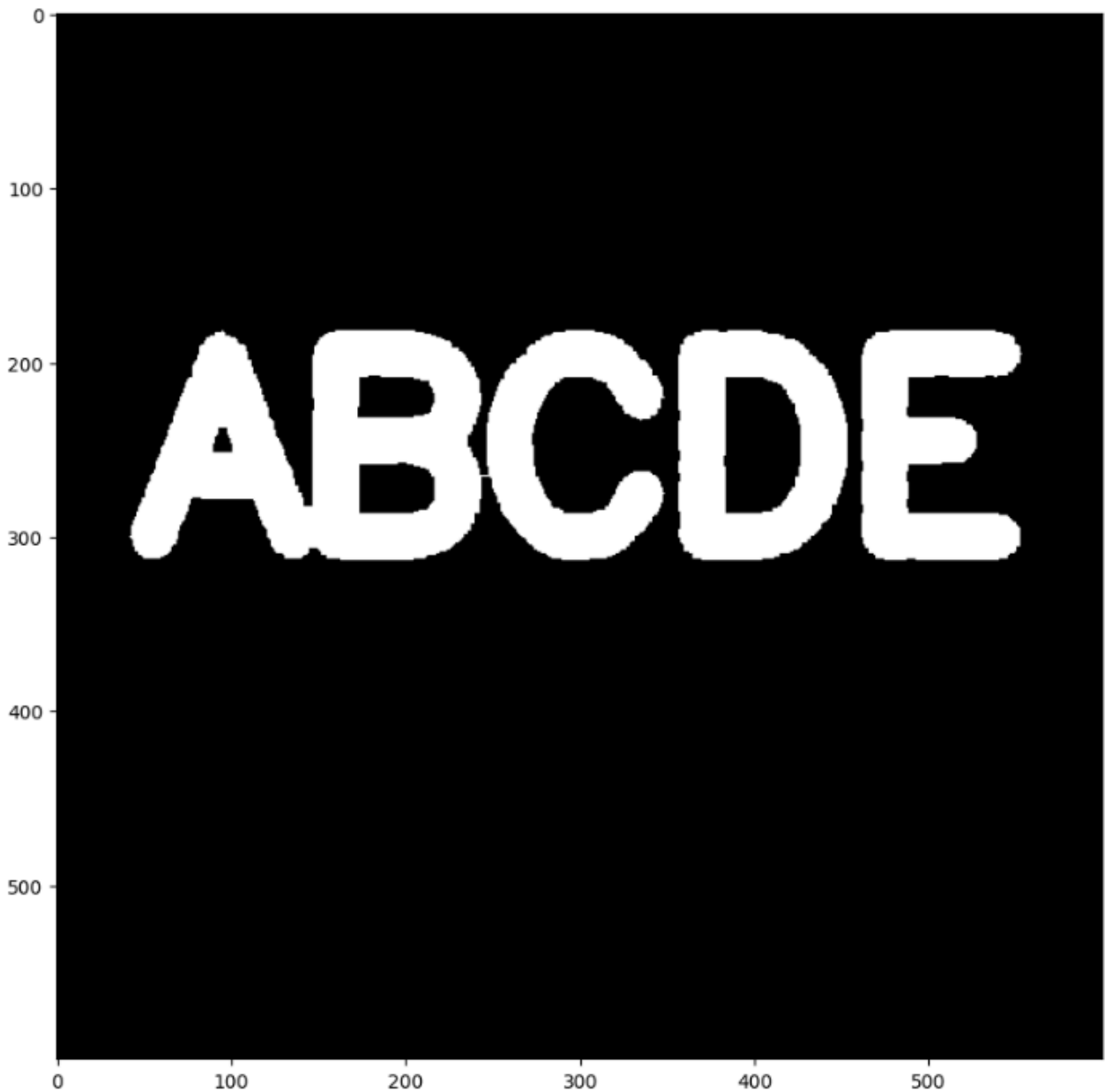
- Opening과 반대로 Dilation 연산을 먼저 적용한 후, Erosion 연산을 적용
- 희색 오브젝트에 있는 작은 검은색 구멍들을 메우는데 사용

크로징 연산을 통한 노이즈 제거

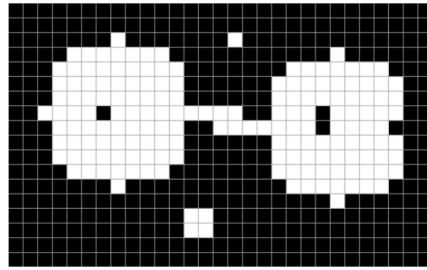
```
closing = cv2.morphologyEx(black_noise_img, cv2.MORPH_CLOSE, kernel)  
display_img(closing)
```

✓ 0.2s

Python



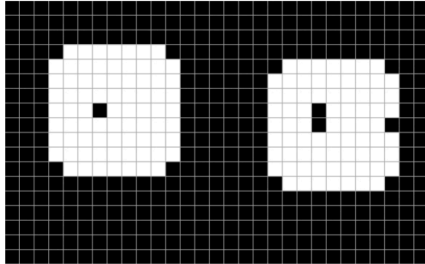
열기와 닫기의 차이



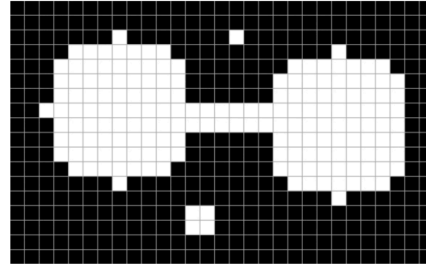
열기

닫기

작은 돌기, 작은
객체가 사라지고,
얇은 연결선이
끊어짐



작은 흠, 작은
홀이 사라지고,
얇은 연결선이
두꺼워짐

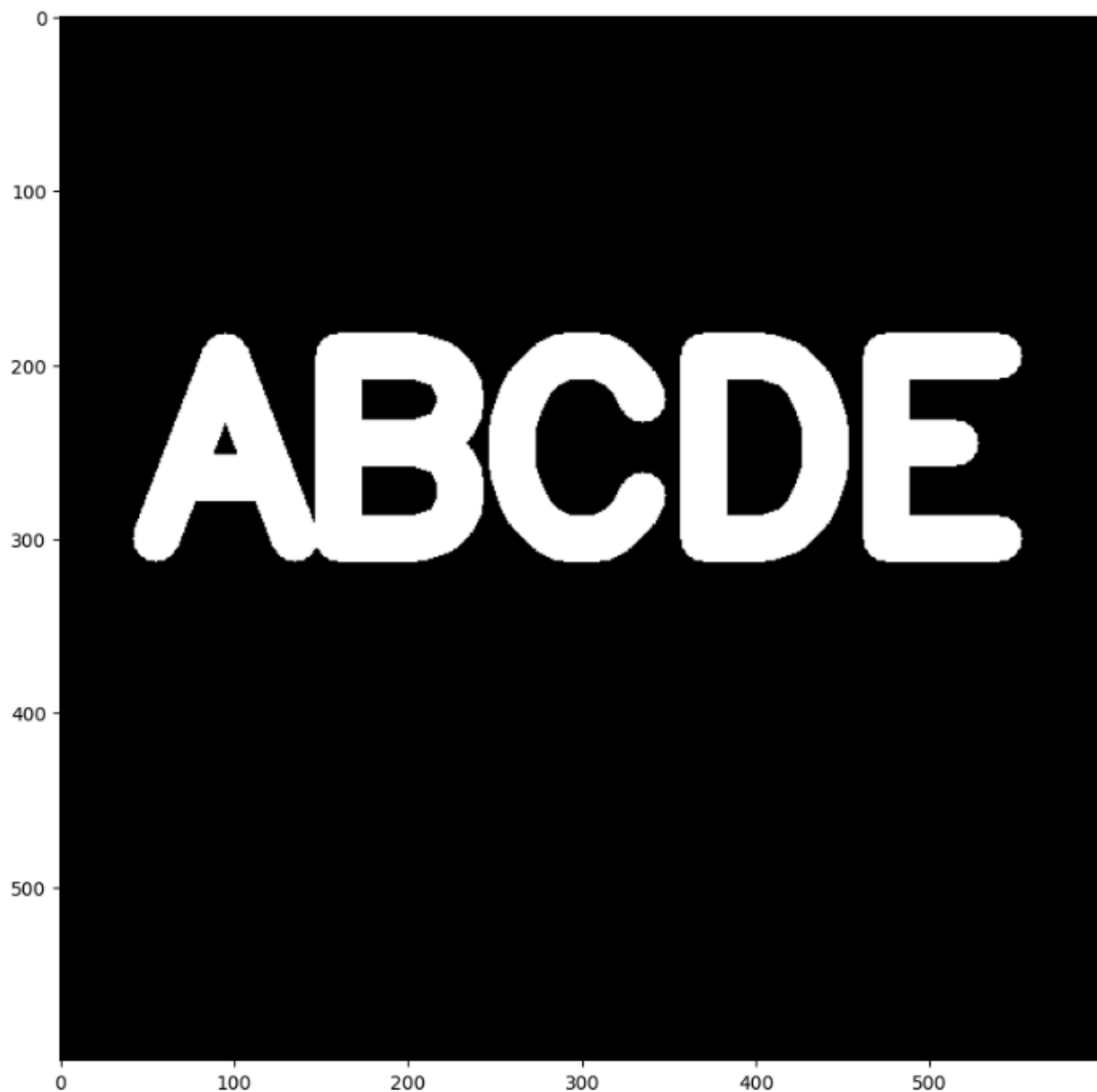


다시 새로 시작

```
img = load_img()  
display_img(img)
```

✓ 0.2s

Python

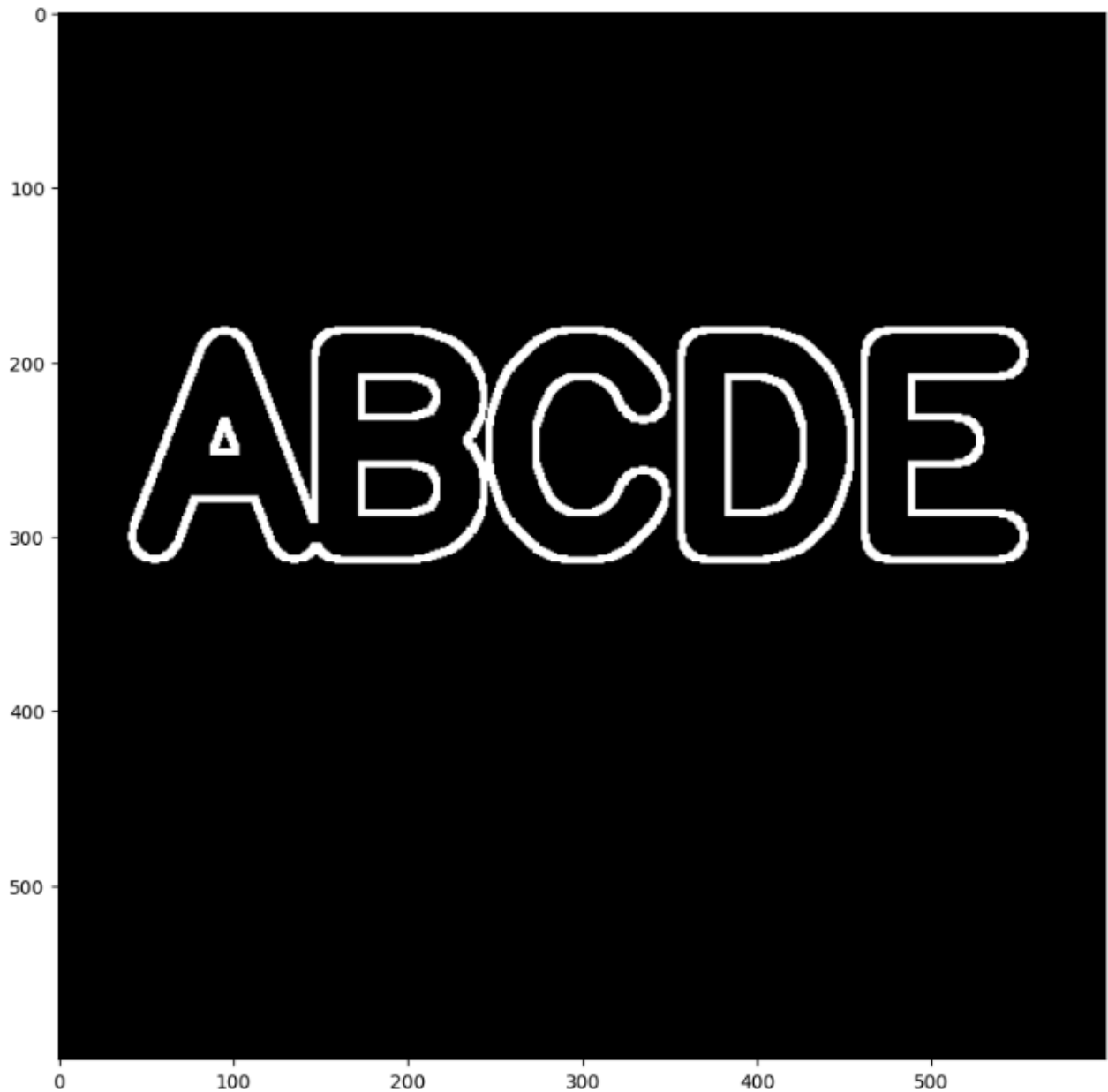


이번에는 그래디언트 연산

```
gradient = cv2.morphologyEx(img, cv2.MORPH_GRADIENT, kernel)
display_img(gradient)
```

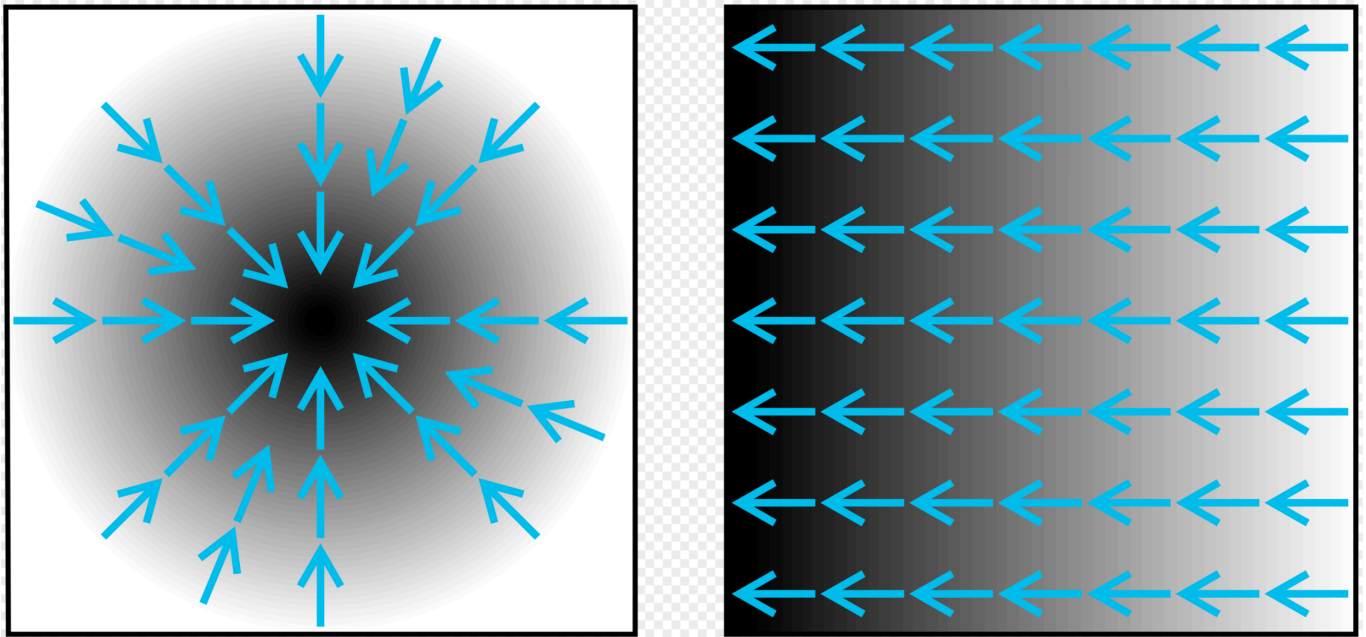
✓ 0.2s

Python



그래디언트

이미지 세상에서 그래디언트는?



그래디언트 중 쇼벨 연산자

$$\mathbf{G}_x = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix} * \mathbf{A} \quad \text{and} \quad \mathbf{G}_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} * \mathbf{A}$$

이미지 그리는

```
def display_img(img):
    fig = plt.figure(figsize=(8, 5))
    ax = fig.add_subplot(111)
    ax.imshow(img, cmap="gray")
```

✓ 0.0s

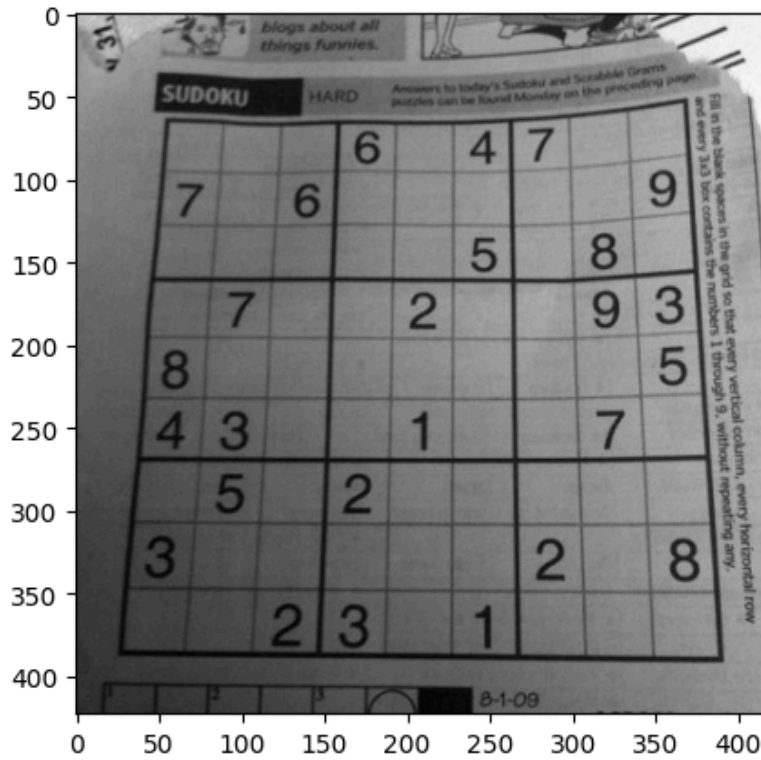
Python

이 이미지 하나 불러와서

```
img = cv2.imread("./data/sudoku.jpg", cv2.IMREAD_GRAYSCALE)
display_img(img)
```

✓ 0.1s

Python

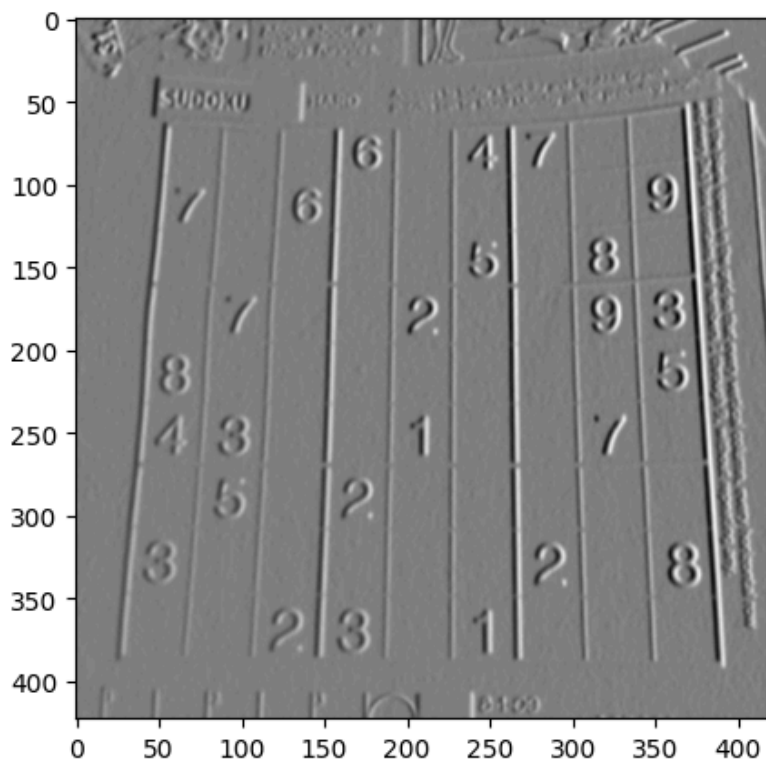


x 방향 쇼벨~

```
sobelx = cv2.Sobel(img, ddepth=cv2.CV_64F, dx=1, dy=0, ksize=5)  
display_img(sobelx)
```

✓ 0.1s

Python

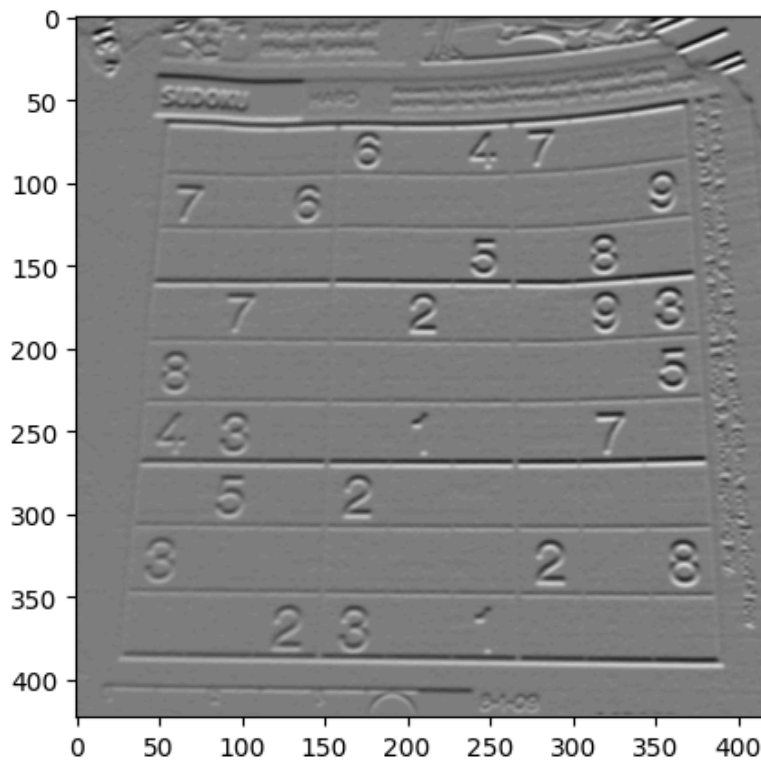


y 방향 그래디언트

```
sobely = cv2.Sobel(img, ddepth=cv2.CV_64F, dx=0, dy=1, ksize=5)  
display_img(sobely)
```

✓ 0.1s

Python

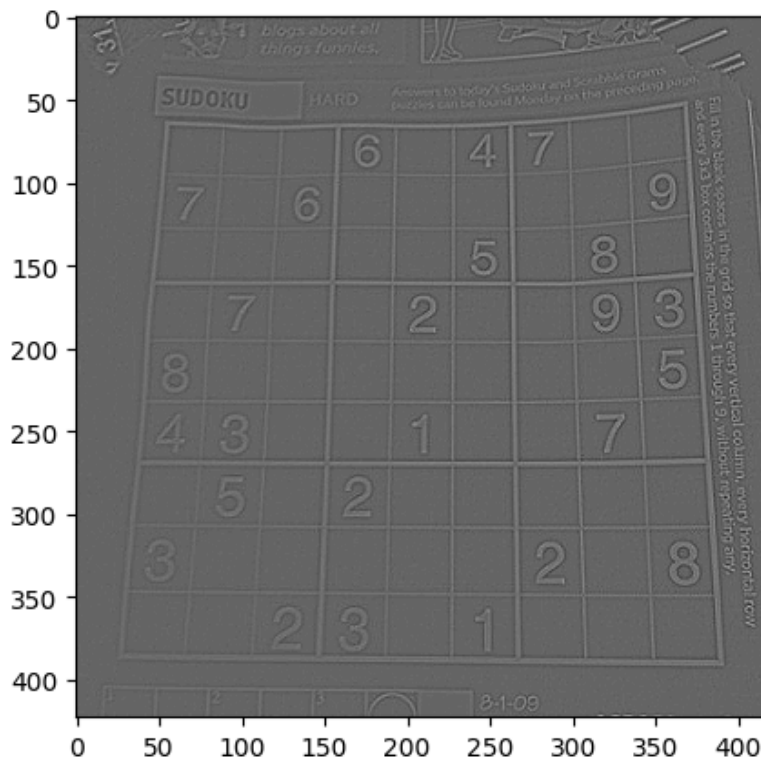


Laplacian 연산

```
1 laplacian = cv2.Laplacian(img, ddepth=cv2.CV_64F)
2 display_img(laplacian)
```

✓ 0.1s

Python

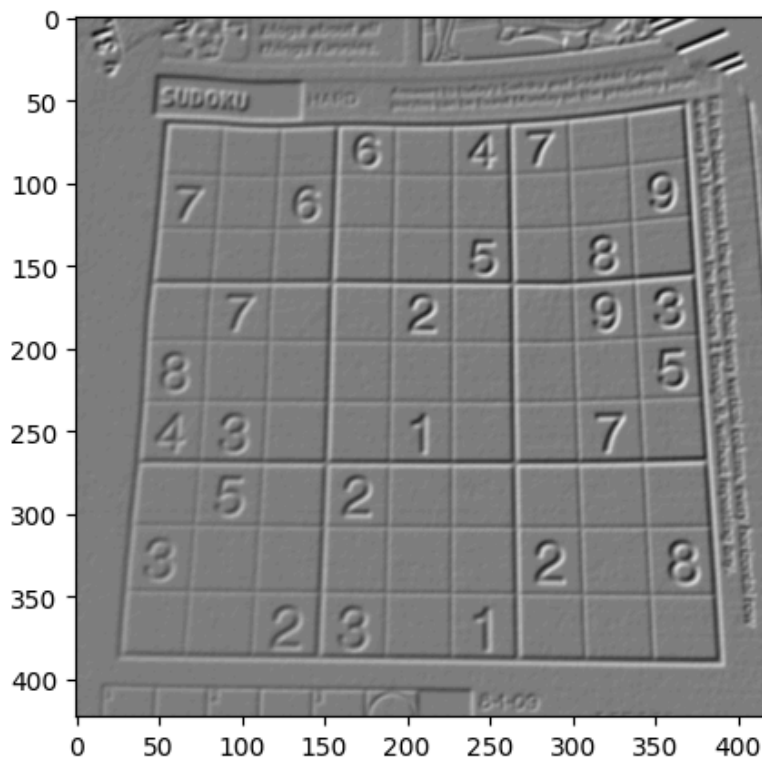


x, y 방향의 그래디언트를 합치기

```
blended = cv2.addWeighted(src1=sobelx, alpha=0.5, src2=sobely, beta=0.5, gamma=0)
display_img(blended)
```

✓ 0.1s

Python



이번에는 threshold 시켜보고

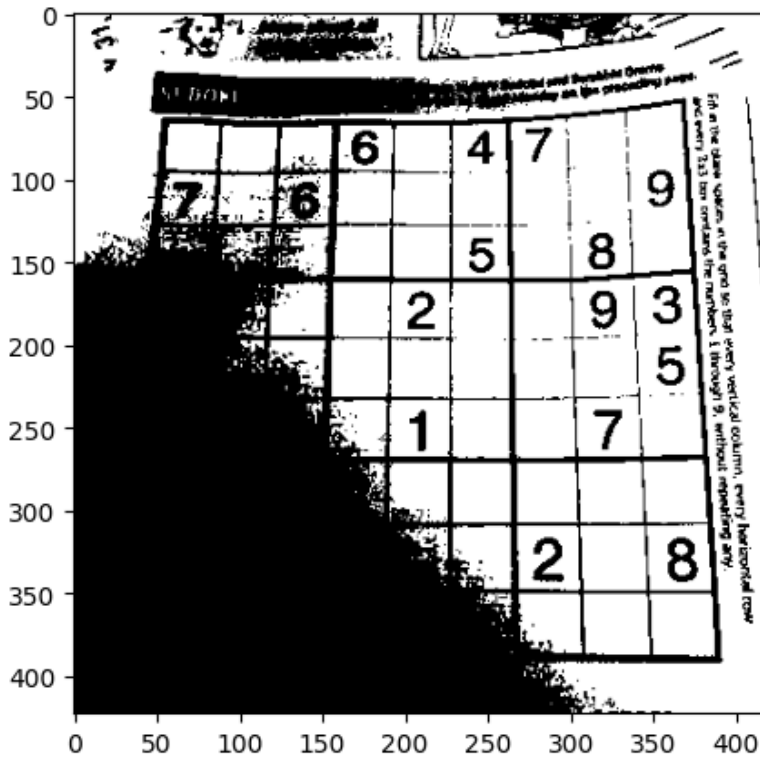

```

1 ret, th1 = cv2.threshold(img, 100, 255, cv2.THRESH_BINARY)
2 display_img(th1)

```

✓ 0.1s

Python

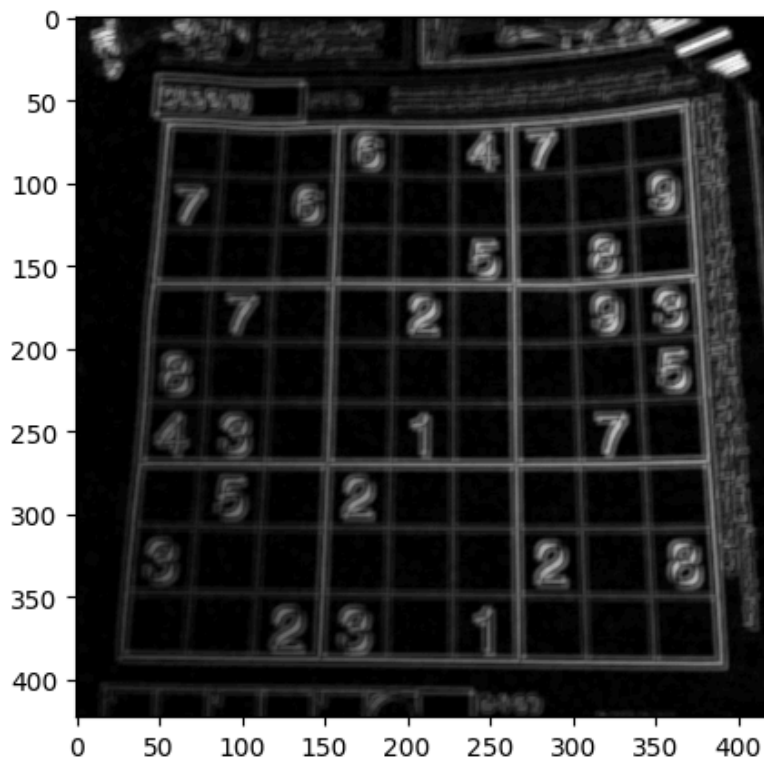


그래디언트 연산으로 모폴로지도 시켜보고

```
kernel = np.ones((4, 4), np.uint8)
gradient = cv2.morphologyEx(blended, cv2.MORPH_GRADIENT, kernel)
display_img(gradient)
```

✓ 0.1s

Python





PinkLAB YouTube Channel

PinkLAB Studio — Robotics, AI, ROS2 and more!

 **Subscribe on YouTube**

https://www.youtube.com/@pinklab_studio